

Arquitecturas Avanzadas

Práctica 1.1

Fecha: 5/02/2025

Alumnos: Paulo Blas Heredia, Almudena Hernández Serrano,
Alejandro Prieto Cepeda

Correos: p.blas@alumnos.upm.es

almudena.hernandez@alumnos.upm.es

alejandro.prieto@alumnos.upm.es

Números de matrícula: br0232, bs0451, bt0428

1. Primera ejecución de hola

```
hola.c - p1.1 - Visual Studio Code

File Edit Selection View Go Run Terminal Help

EXPLORER
  C:\hola.c
  P11
  C:\costemc.c
  C:\costeop.c
  C:\helloWorld.c
  E hola
  E hola_hosts.txt
  C hola.c
  Makefile
  E pingpong_hosts.txt
  E pingpong.c
  C psendrec.c

C:\hola.c
17 void esclavo(int yo) {
44     if (err != MPI_SUCCESS) {
45         fprintf(stderr, "Error: MPI_Send falló en esclavo\n");
46         MPI_Abort(MPI_COMM_WORLD, err);
47     }
48 }
49
50 //-----
51 void maestro(int numEsclavos) {
52     int i;
53     char buffer[LONG_BUFFER] = {0};
54     long int tiempo[TIME_ELEMENTS];
55     MPI_Status estado;
56
57     // Obtener el nombre del host del maestro
58     gethostname(buffer, LONG_BUFFER);
59     printf("Maestro ejecutándose en %s\n", buffer);
60
61     // Recibir mensajes de los esclavos
62     for (i = 0; i < numEsclavos; i++) {
63         int err = MPI_Recv(buffer, LONG_BUFFER, MPI_BYTE, MPI_ANY_SOURCE, TAG_MESSAGE,
64                             MPI_COMM_WORLD, &estado);
65         if (err != MPI_SUCCESS) {
66             fprintf(stderr, "Error: MPI_Recv falló en maestro\n");
67             MPI_Abort(MPI_COMM_WORLD, err);
68         }
69     }
70 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
* arquava52@4401-12:~/p1.1$ mpiexec -n 9 hola
Maestro ejecutándose en 4401-12
Del proceso 4: Hola, desde 4401-12
Del proceso 5: Hola, desde 4401-12
Del proceso 2: Hola, desde 4401-12
Del proceso 3: Hola, desde 4401-12
Del proceso 1: Hola, desde 4401-12
Del proceso 6: Hola, desde 4401-12
Del proceso 8: Hola, desde 4401-12
Del proceso 7: Hola, desde 4401-12
Tiempo del Proceso[1] = 6.000095 s
Tiempo del Proceso[2] = 7.000180 s
Tiempo del Proceso[3] = 8.000098 s
Tiempo del Proceso[4] = 9.000128 s
Tiempo del Proceso[5] = 10.000126 s
Tiempo del Proceso[6] = 11.000067 s
Tiempo del Proceso[7] = 12.000059 s
Tiempo del Proceso[8] = 13.000068 s
arquava52@4401-12:~/p1.1$
```

Explica por qué los primeros mensajes de Hola ... desde los esclavos salen en desorden y los últimos de Tiempo ... lo hacen en orden

Porque desde el método maestro cuando recibe los mensajes de los esclavos en “MPI_Recv” indica “MPI_ANY_SOURCE” indica que puede recibir mensajes de cualquier proceso en cualquier orden.

Sin embargo, a la hora de recibir los datos de tiempo de los esclavos, se indica “i + 1”, indicando un orden.

Además, en el método de esclavo, el sleep añade variabilidad: “sleep(yo + 5)”, lo que provoca que no se envíen en orden.

2. Ejecución de hola en varias máquinas

The image shows a screenshot of the Visual Studio Code editor interface. The top status bar indicates the date and time: "mié, 12 de feb, 17:20". The main window title is "hola_hosts.txt - p1.1 - Visual Studio Code".

The Explorer sidebar on the left shows the file structure of the project "p1.1":

- costem.c
- costeop.c
- helloWorld.c
- hola
- hola_hosts.txt (selected)
- hola.c
- Makefile
- pingpong_hosts.txt
- pingpong.c
- psendrec.c
- salida.txt

The main editor area displays the content of "hola_hosts.txt":

```
1 4401-12:4 # Primeros 4 procesos en el pc1
2 4401-13:4 # Siguientes 4 procesos en el pc2
3
```

The TERMINAL panel at the bottom shows the execution of the program. It starts with a message: "Now try logging into the machine, with: 'ssh 'arquava52@4401-10'' and check to make sure that only the key(s) you wanted were added."

The terminal output shows the following commands and results:

```
arquava52@4401-12:~$ cd p1.1/
arquava52@4401-12:~/p1.1$ ssh 4401-13 "mkdir p1.1"
arquava52@4401-12:~/p1.1$ scp hola 4401-13:p1.1
hola                                100% 22KB 11.4MB/s 00:00
arquava52@4401-12:~/p1.1$ mpxexec -n 9 -f hola hosts.txt hola
Maestro ejecutándose en 4401-12
Del proceso 1: Hola, desde 4401-12
Del proceso 2: Hola, desde 4401-12
Del proceso 3: Hola, desde 4401-12
Del proceso 8: Hola, desde 4401-12
Del proceso 4: Hola, desde 4401-13
Del proceso 5: Hola, desde 4401-13
Del proceso 7: Hola, desde 4401-13
Del proceso 6: Hola, desde 4401-13
Tiempo del Proceso[1] = 0.000095 s
Tiempo del Proceso[2] = 7.000094 s
Tiempo del Proceso[3] = 8.000103 s
Tiempo del Proceso[4] = 9.001938 s
Tiempo del Proceso[5] = 10.001608 s
Tiempo del Proceso[6] = 11.001610 s
Tiempo del Proceso[7] = 12.001612 s
Tiempo del Proceso[8] = 13.000074 s
arquava52@4401-12:~/p1.1$
```

3. Tiempos de comunicación

Actividades Visual Studio Code

mié, 12 de feb, 18:41 •

pingpong.c - p1.1 - Visual Studio Code

File Edit Selection View Go Run Terminal Help

EXPLORER

- PL1
- costemp.c
- costemp.c
- helloWorld
- helloWorld.c
- hola
- hola_hosts.txt
- hola.c
- Makefile
- pingpong
- pingpong_hosts.txt
- pingpong.c
- psendrec.c
- salida.txt

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Tiempo medio = 0.006652500 milisegundos
arqava5284401-12-~/p1.1$ make pingpong
mpicc -Wall -g -DDEBUG pingpong.c -o pingpong
arqava5284401-12-~/p1.1$ mpxexec -n 2 pingpong
0.227000000 0.031000000 0.024000000 0.023000000 0.023500000
0.024500000 0.024000000 0.023000000 0.023000000 0.023500000
0.023500000 0.024000000 0.023500000 0.023500000 0.023500000
0.024500000 0.023000000 0.023500000 0.024500000 0.023500000
0.023000000 0.024000000 0.024000000 0.024500000 0.023500000
0.024000000 0.023500000 0.023000000 0.023000000 0.023500000
0.023000000 0.023000000 0.022500000 0.023000000 0.023000000
0.023500000 0.023000000 0.023000000 0.023000000 0.023000000
0.022500000 0.023000000 0.023000000 0.022500000 0.022500000
0.023000000 0.022500000 0.023000000 0.023000000 0.023000000
0.022500000 0.022500000 0.024500000 0.022500000 0.023000000
0.022500000 0.022500000 0.027000000 0.023000000 0.022500000
0.023000000 0.023000000 0.022500000 0.022000000 0.021500000
0.022000000 0.021500000 0.021500000 0.021500000 0.022000000
1.749000000 0.023500000 0.021500000 0.021500000 0.022000000
0.021500000 0.021500000 0.070000000 0.022000000 0.021500000
0.021500000 0.021500000 0.021500000 0.021500000 0.021500000
0.021500000 0.021500000 0.021500000 0.021500000 0.021500000
0.022000000 0.021500000 0.021500000 0.021500000 0.022000000
0.022000000 0.021500000 0.021500000 0.021500000 0.021500000
0.022000000 0.021500000 0.022000000 0.022000000 0.021500000
0.021500000 0.021500000 0.022000000 0.022000000 0.021500000
0.021500000 0.022500000 0.022000000 0.022500000 0.021500000
0.062000000 0.022500000 0.021500000 0.021500000 0.021500000
0.021500000 0.022000000 0.021500000 0.021500000 0.021500000
0.021500000 0.022000000 0.022000000 0.022000000 0.022000000
0.021500000 0.022000000 0.021500000 0.035500000 0.022000000
0.022000000 0.022500000 0.021500000 0.021500000 0.022000000
0.021500000 0.021500000 0.021000000 0.021000000 0.021000000
0.021000000 0.021500000 0.021000000 0.021000000 0.020500000
0.021500000 0.021500000 0.021000000 0.021000000 0.021000000
0.021500000 0.022000000 0.021500000 0.021500000 0.021000000
0.022000000 0.021000000 0.021500000 0.021000000 0.021500000
0.021500000 0.021500000 0.021000000 0.021500000 0.021500000
0.021000000 0.021000000 0.021000000 0.021000000 0.020500000
0.020500000 0.021000000 0.021000000 0.020500000 0.021000000
0.020500000 0.021000000 0.020500000 0.021000000 0.021000000
0.021000000 0.021000000 0.021500000 0.021500000 0.021000000
0.021000000
Tiempo medio = 0.032527500 milisegundos
arqava5284401-12-~/p1.1$
```

hola.c

```
17 void esclavo(int y) {
26     gethostname(buffer + strlen(buffer), LONG_BUFFER - strlen(buffer));
```

bash

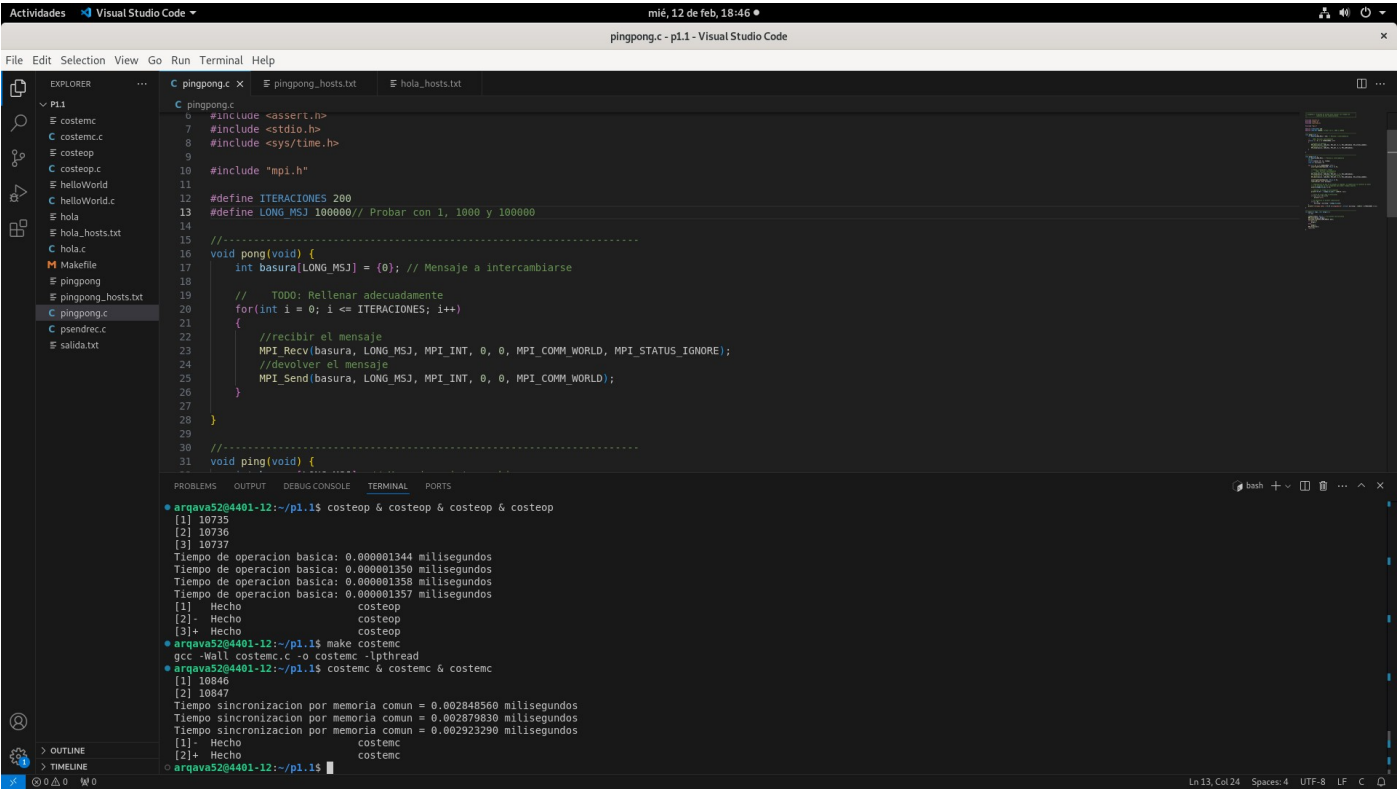
	En mismo PC (ms)	Entre dos PCs (ms)	Entre dos PCs (Mbps)
ping	0,036	0,187	-
pingpong 1 int	0,000272500	0,00023	-
pingpong 1000 int	0,0006825	0,0006525	49042,15
pingpong 100000 int	0,0215075	0,0325275	9837,83

(incluir fichero pingpong)

4. Tiempos de operaciones

Rellenar las dos tablas y captura de pantalla de las ejecuciones.

	Tiempo (ms)
costeop	0,000001344
costemc	0,00284856



costeop	costemc	pingpong 1 int mismo PC	pingpong 1 int entre dos PCs
1	2119,46	202,75	171,13

Código del ping pong

```
//-----+
// pingpong.c: Programa de prueba para evaluar los tiempos de      |
//      latencia de las comunicaciones.      |
//-----+

#include <assert.h>
#include <stdio.h>
#include <sys/time.h>

#include "mpi.h"

#define ITERACIONES 200
#define LONG_MSJ 1 // Probar con 1, 1000 y 100000

//-----+
void pong(void) {
    int basura[LONG_MSJ]; // Mensaje a intercambiarse
    int i;

    for(i = 0; i <= ITERACIONES; i++)
    {
        //recibir el mensaje enviado por proceso 0
        MPI_Recv(basura, LONG_MSJ, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        //devolver el mensaje al proceso 0
        MPI_Send(basura, LONG_MSJ, MPI_INT, 0, 0, MPI_COMM_WORLD);
    }
}

//-----+
void ping(void) {
    int basura[LONG_MSJ]; // Mensaje a intercambiarse
    int i;
    struct timeval t0, t1, tiempo;
    long int microseg = 0;

    for (i = 0; i <= ITERACIONES; i++) {
```

```

assert(gettimeofday(&t0, NULL) == 0);

// Envio y recepcion a "pong"
// envio del mensaje al proceso 1
MPI_Send(basura, LONG_MSJ, MPI_INT, 1, 0, MPI_COMM_WORLD);
// recepcion de la respuesta del proceso 1
MPI_Recv(basura, LONG_MSJ, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

assert(gettimeofday(&t1, NULL) == 0);
timersub(&t1, &t0, &tiempo);

// Comprobacion de que no ha pasado un segundo: se supone que la latencia es menor
// y el programa no está preparado para medir tiempos mayores
assert(tiempo.tv_sec == 0);

// Imprimir el tiempo en milisegundos
printf("%7.9f ", tiempo.tv_usec / (1000.0 * 2));

// Salto de línea cada 5 iteraciones
if (((i + 1) % 5) == 0)
    printf("\n");

// Se desprecia la primera comunicacion
if (i > 0)
    microseg = microseg + tiempo.tv_usec;
}
printf("\nTiempo medio = %7.9f milisegundos\n", (float) microseg / (1000.0 * ITERACIONES * 2));
}

//-----
int main(int argc, char *argv[]) {
    int yo;

    setbuf(stdout, NULL); // Sin buffers de escritura
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &yo);
    if (yo == 0)
        ping();
    else

```

```
    pong();  
    MPI_Finalize();  
    return 0;  
}
```